

SAIR

Spatial AI & Robotics Lab

CSE 473/573

L19: DEEP LEARNING

Chen Wang

Spatial AI & Robotics Lab

Department of Computer Science and Engineering

University at Buffalo The State University of New York

Many Slides from Kris Kitani

Classic Classification Pipeline

Input → **encoder** → **decoder** → **output**

image(s)

Extract features

Build descriptor

Optimizer

semantic label

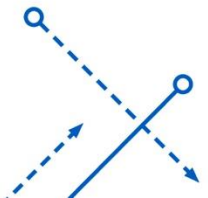
e.g., SIFT, Bag of visual words

e.g., KNN, SVM, Naive Bayes

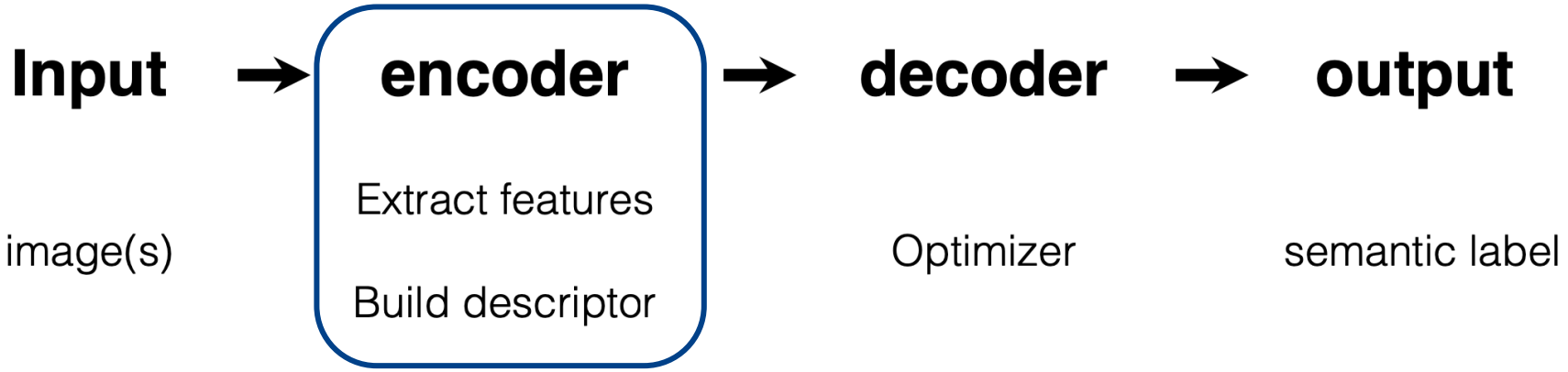
Designed by you

Painful.

Learned from the data



Classic Classification Pipeline



Can you think of a way to learn this part too?

*How can we learn it all '**end to end**?'*

End-to-end Learning

Input → **encoder** → **decoder** → **output**

image(s)

Extract features

Build descriptor

Optimizer

semantic label

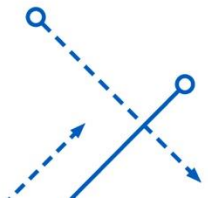
Convolutional Neural Layers

Fully connected layers

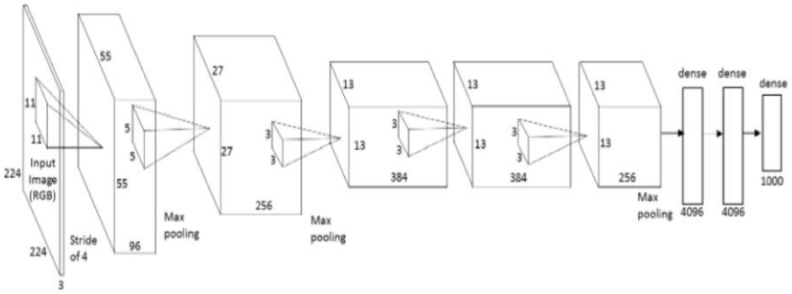
Deep Network

Use gradient descent to learn everything

(except for the architecture, which you have to design. painful.)



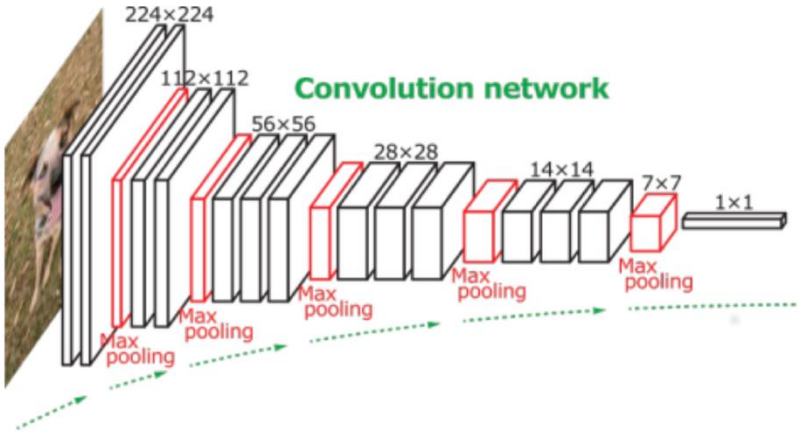
Popular CNN architectures



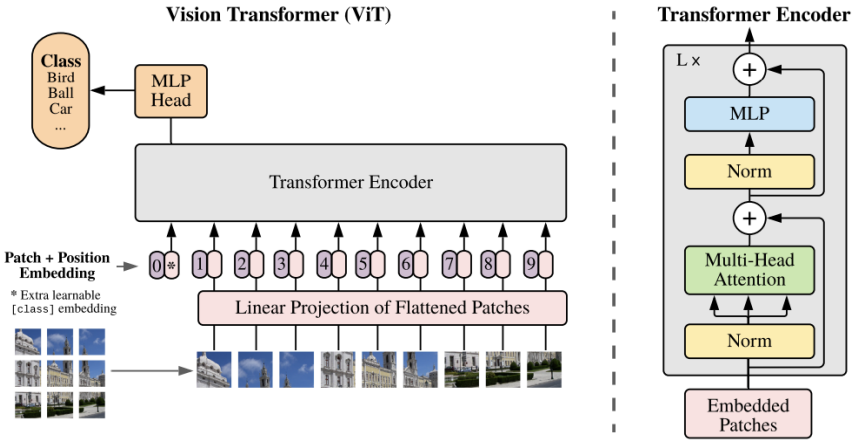
AlexNet



ResNet



VGG



Vision Transformer

The origin …

Y. Le Cun, L. D. Jackel, B. Boser, J. S. Denker, H. P. Graf, I. Guyon, D. Henderson, R. E. Howard, and W. Hubbard. Handwritten digit recognition: Applications of neural net chips and automatic learning. *IEEE Communication*, pages 41–46, November 1989. invited paper.

Handwritten Digit Recognition: Applications of Neural Network Chips and Automatic Learning

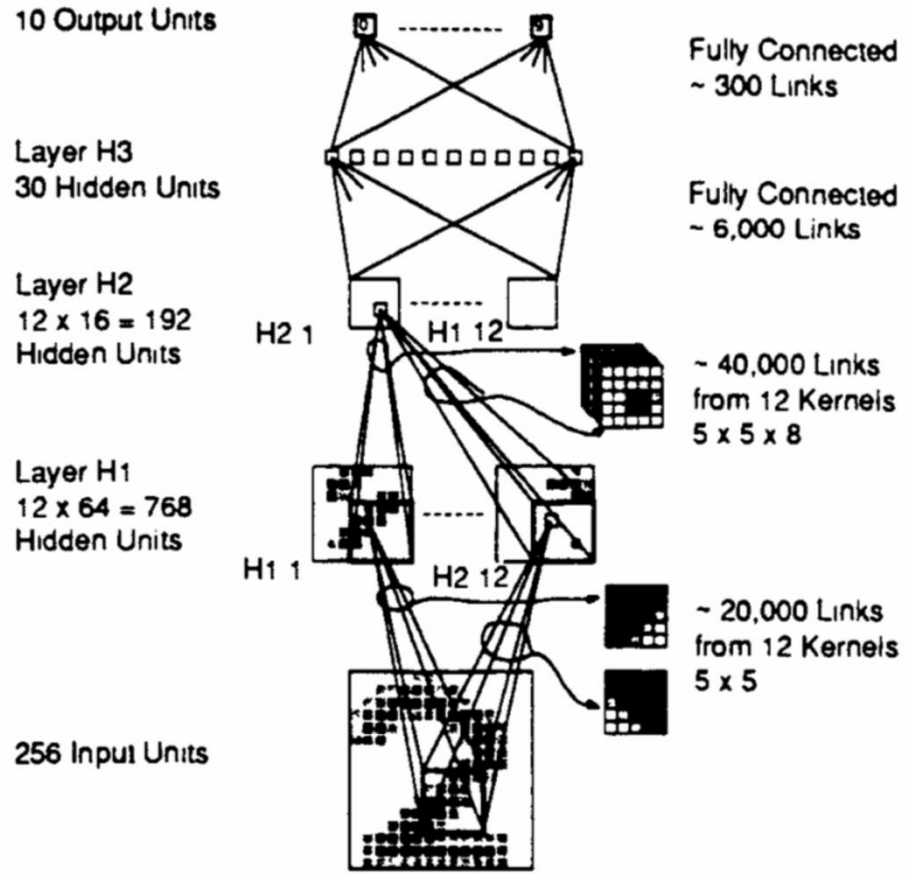
Y. Le Cun
L. D. Jackel
B. Boser
J. S. Denker
H. P Graf

I. Guyon
D. Henderson
R. E. Howard
W. Hubbard

THIS ARTICLE DESCRIBES TWO NEW METHODS for achieving handwritten digit recognition. The task of handwritten digit recognition was chosen for investigation not only because it has considerable practical interest, but because it is relatively well-defined and is sufficiently complex to constitute a meaningful test of connectionist methods.

Simple classification techniques applied to pixel images do not provide high recognition rates because systems based on these techniques contain little prior knowledge about the topology of the task. Knowledge can be built into the system by changing the representation of a digit from a pixel image to a predefined feature description. The first of our methods implements this idea by performing feature extraction with a neural network chip. The feature representation can then be used by a relatively simple classifier, consisting of a two-layer network

is highly test-set dependent. A system may successfully recognize 99% of test data consisting of well-formed digits but score only 80% when confronted with the poorly-formed digits that are both routinely produced and easily recognized by people. We choose to perform our experiments on a rather difficult data set: isolated handwritten digits that were taken from postal zip codes. The zip code images were collected by the U.S. Postal Service from envelopes that passed through the Buffalo, NY Post Office. A postal service contractor converted the original zip code images to binary images, and segmented the digits; that is, disaggregated them into five disjointed images, one for each digit of the zip code. The resulting database consists of 9,298 binary images of isolated digits, 7,291 of which are used as the training set, while the remaining 2,007 are used as the test set. Most of the images are fairly clean; however, a significant fraction are very blotchy or incomplete. The latter defect



1989!

This led to a famous model known as...

LeNet-5

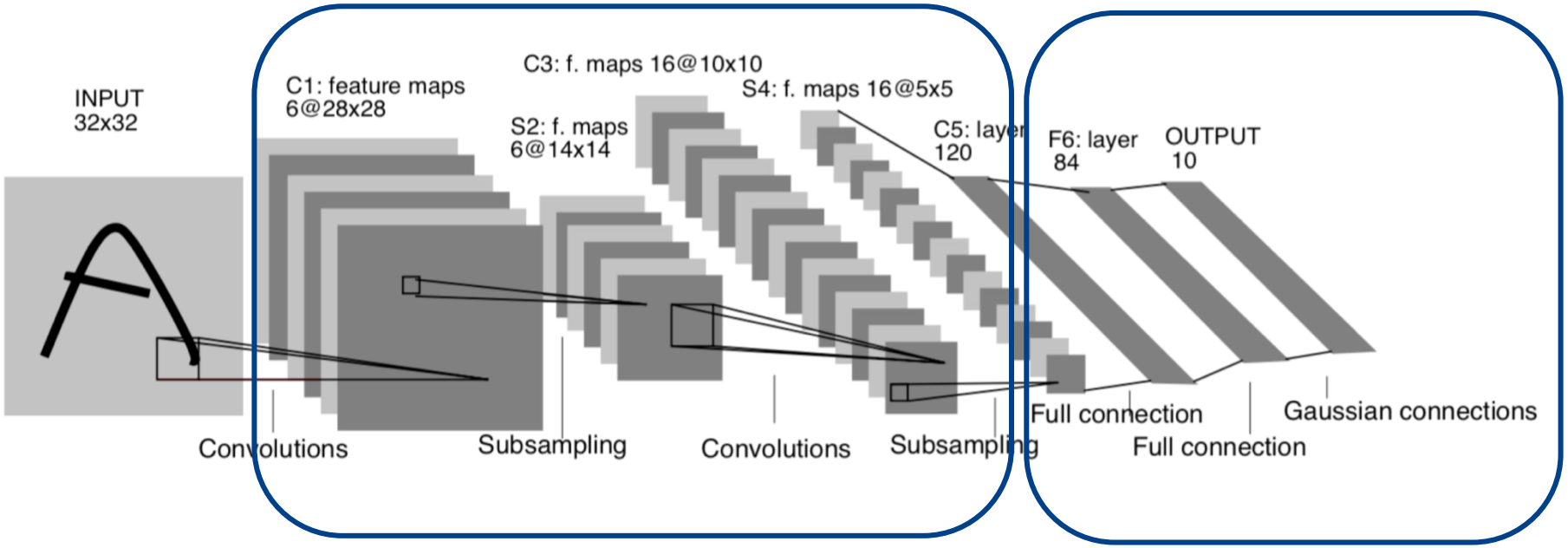


Image input

Encoder

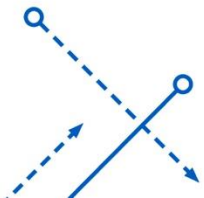
Extracts a representational vector of the image.

Takes place of SIFT and BOW

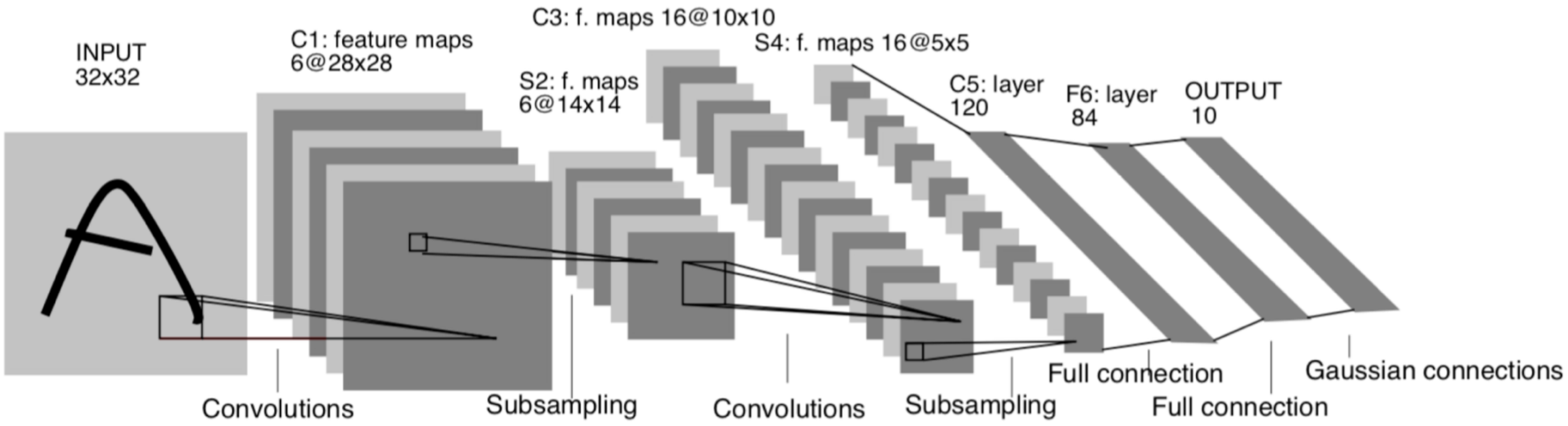
Decoder

Maps vector to categorical probabilities.

Just an MLP

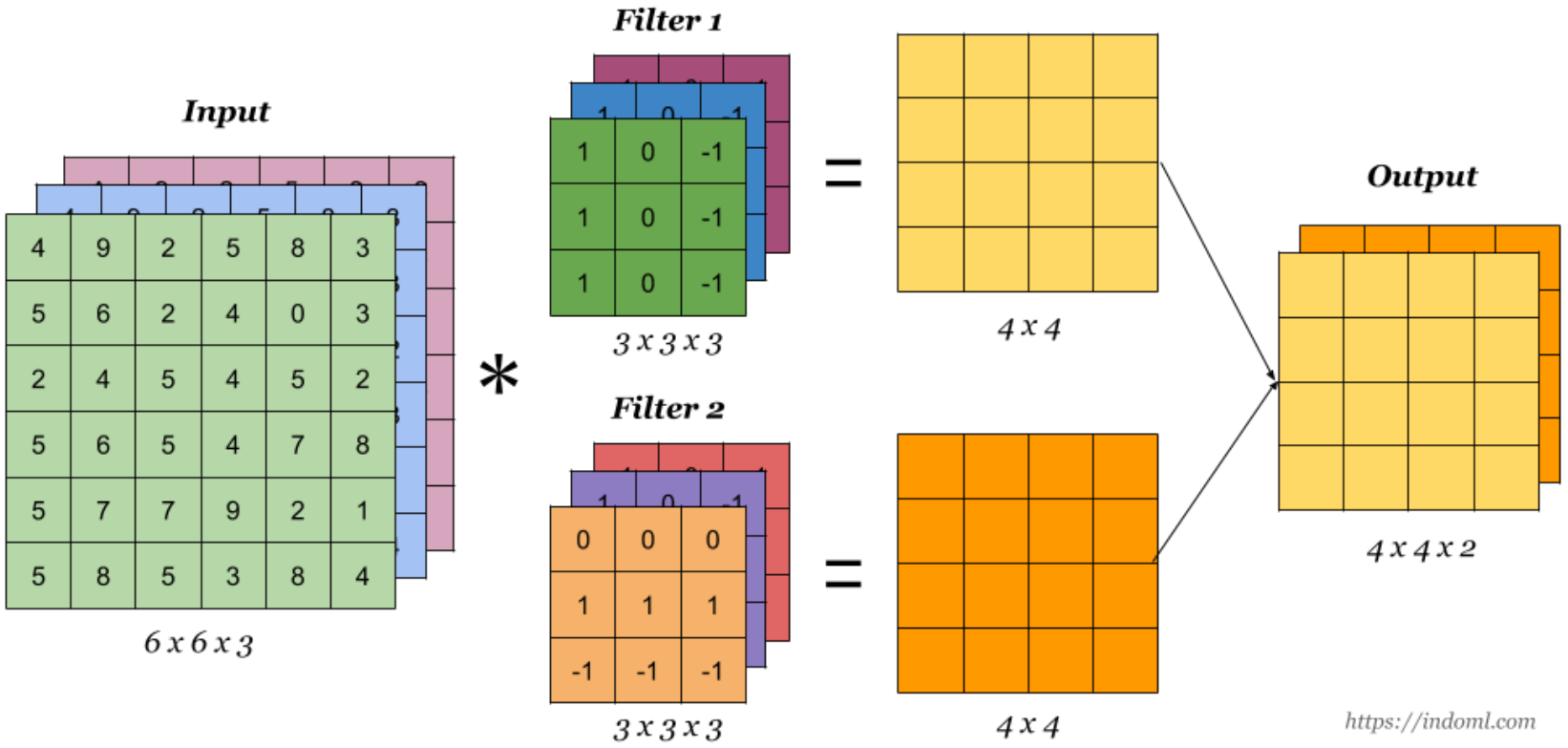


LeNet-5



- Input image size is fixed, what do you do if image is too big?
  - Convolutions with small receptive field.
- How do convolution layer work?

Convolutional Layer



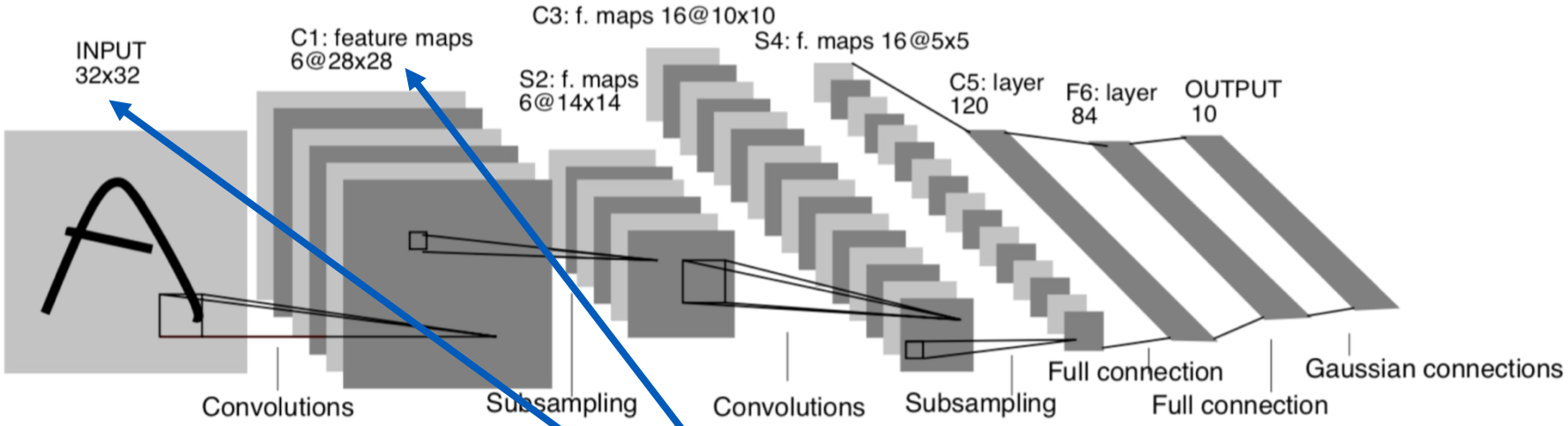
Examples:

```
>>> # With square kernels and equal stride
>>> filters = torch.randn(8, 4, 3, 3)
>>> inputs = torch.randn(1, 4, 5, 5)
>>> F.conv2d(inputs, filters, padding=1)
```

What is shape of the output in the left example?

(1, 8, 5, 5)

Feature map



- If the feature map is 28 x 28, what is the size of the convolution kernel?
- If you don't assume border padding, then 5 x 5.
  - Can't compute convolution for 2 pixels on border.
- Why not use a bigger kernel size?

Feature similarity

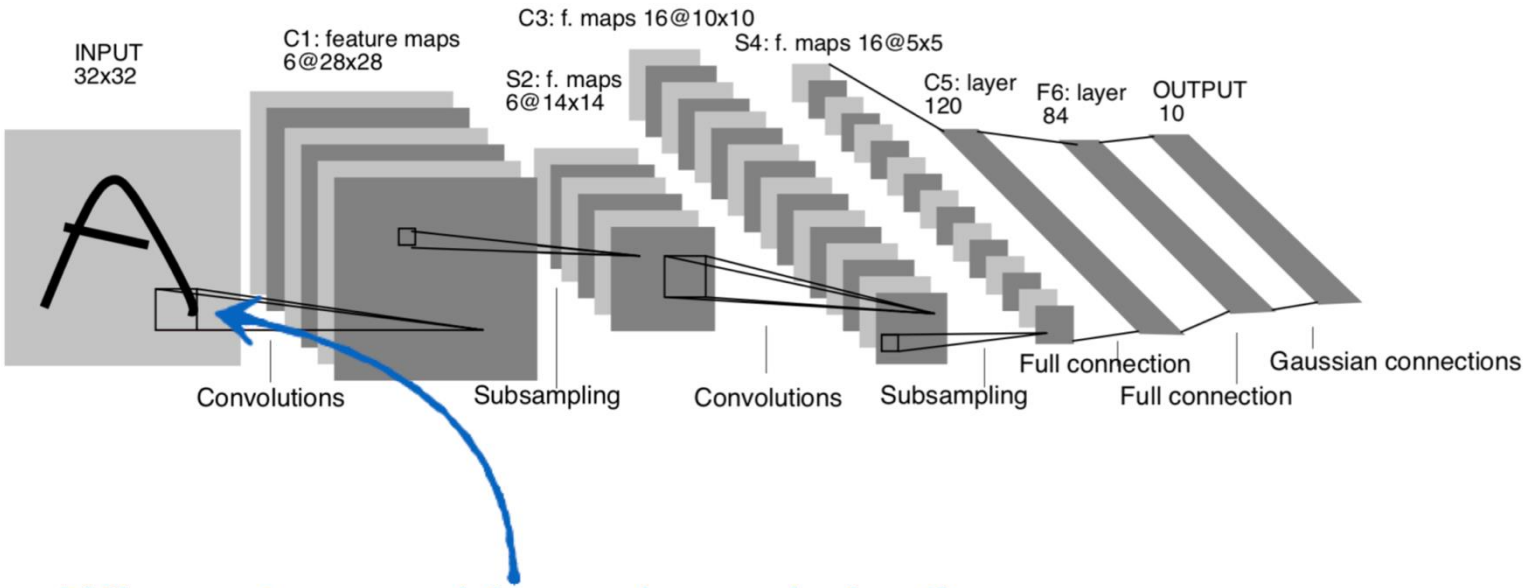


Feature similarity



.. features also similar across objects

Kernel size



Why not use a bigger kernel size?

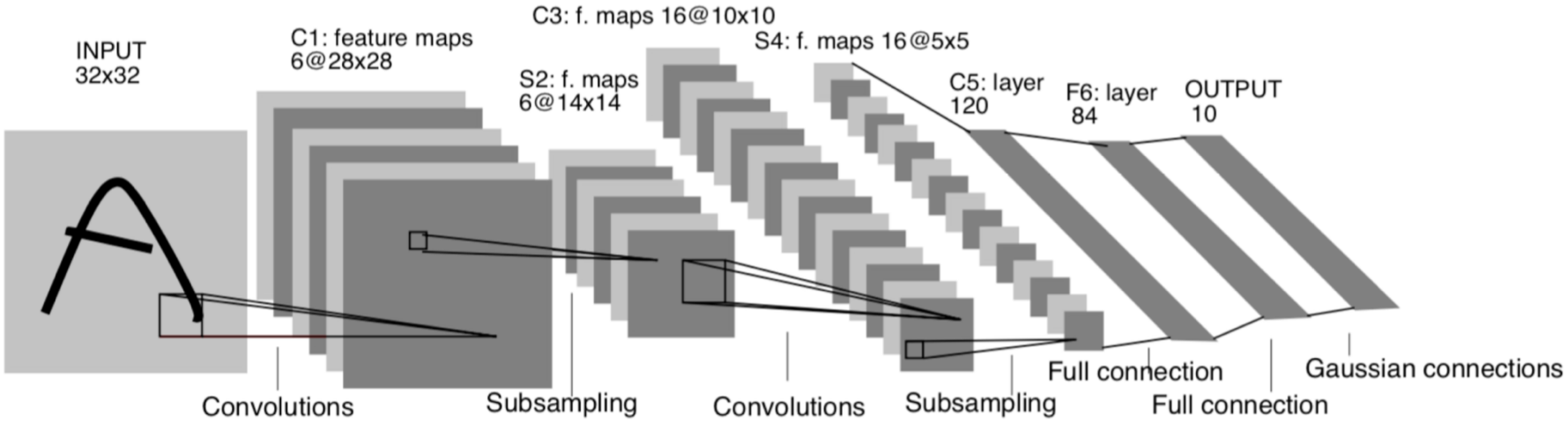
A small kernel can capture common primitive patterns across a wide range of images



Can be reused over the entire image (shared weights)

How many kernels did we use here? (# output channel)

Subsampling



Subsampling reduces the size of feature maps

(actually average pooling with scale and bias, sigmoid function)

$$h[i,j] = \text{sigmoid}\left(\alpha \cdot \sum_{u,v} g[u,v] f[i-u, j-v] - \beta\right)$$

scale; 'Average Pooling'; bias; Box filter; Feature map

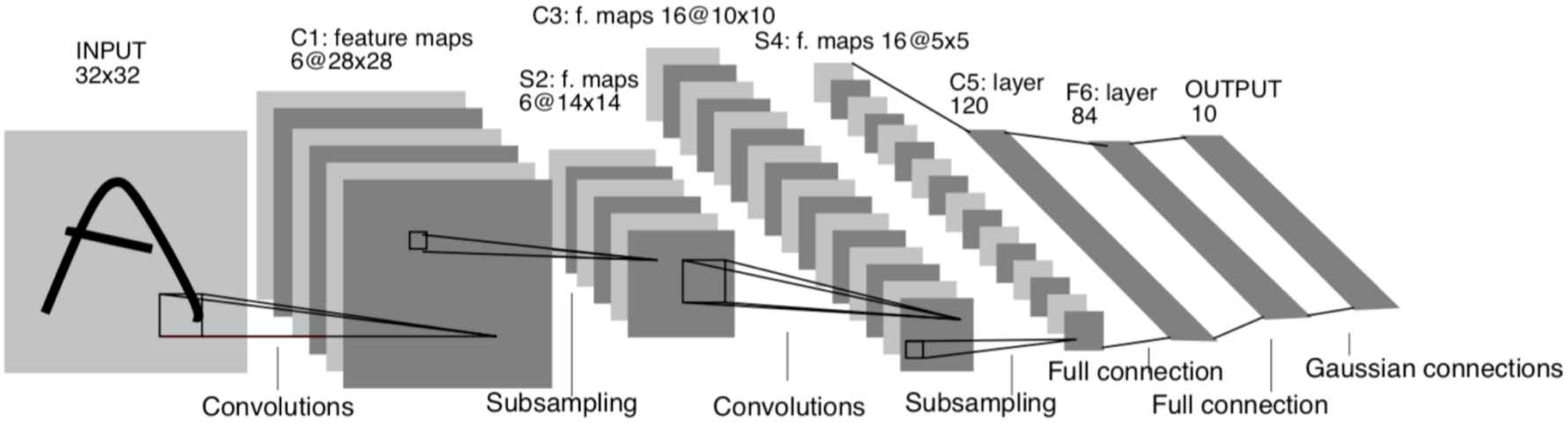
What is the step size and receptive field of subsampling?

Step size: 2, Receptive field: 2 x 2

How does subsampling affect the image representation?

'Blurs' the image (Low pass filter).

Multi-channel convolution



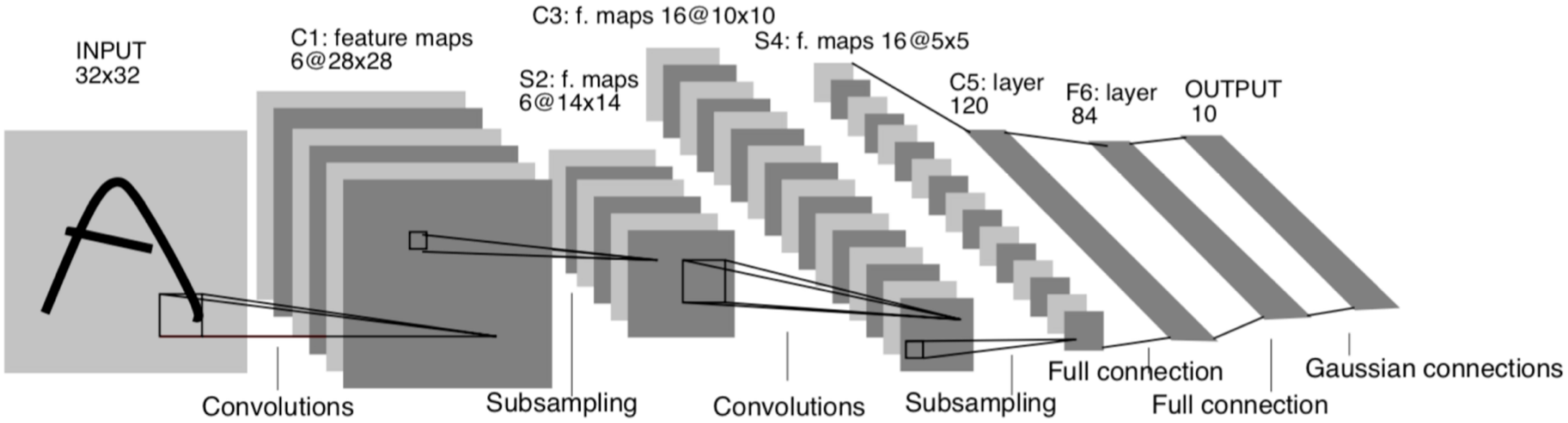
Convolution is now multichannel

$$h[i,j] = \sum_k \sum_u \sum_v g[u,v,k]\ f[i-u, j-v, k]$$

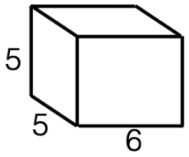
Is this a 3D convolution?

No. Why?

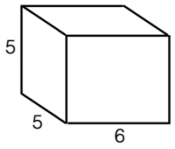
Convolutional filter



- What is the size of the convolutional filter at C3?
  - 6×5×5

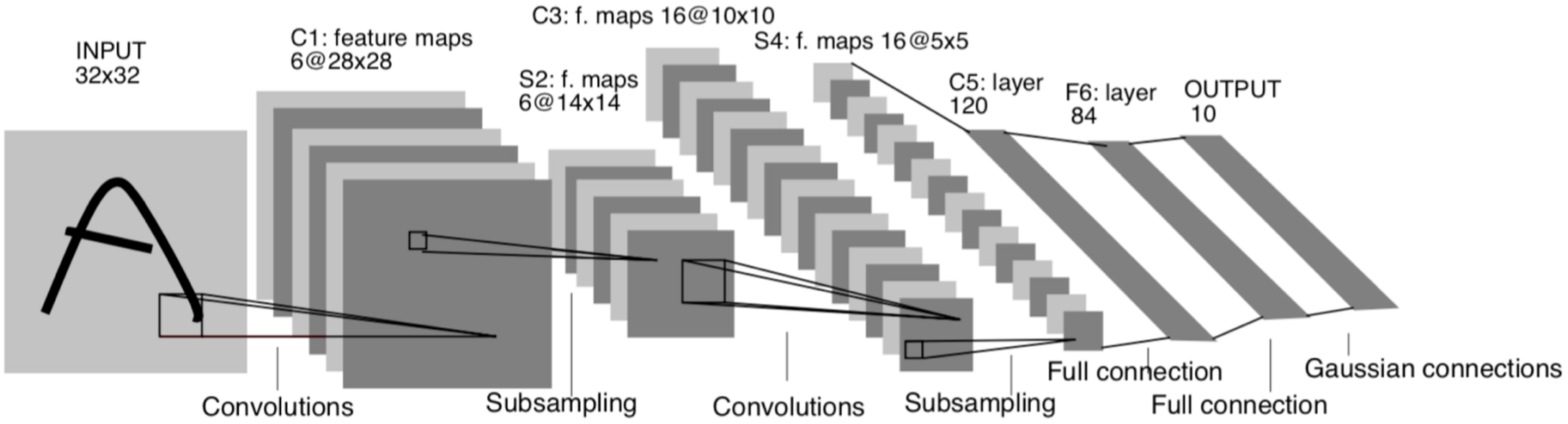


- How many features maps for one multi-channel convolution?

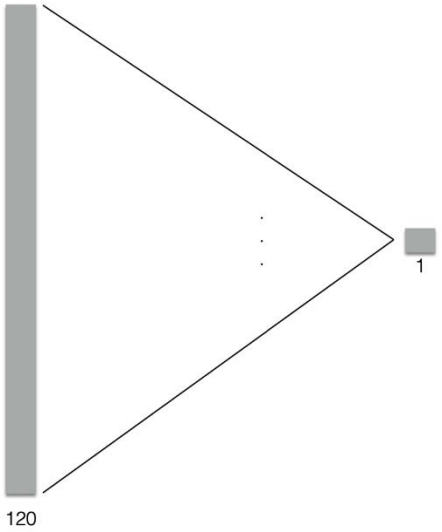




Fully connection



One huge perceptron!
(don't forget the bias)



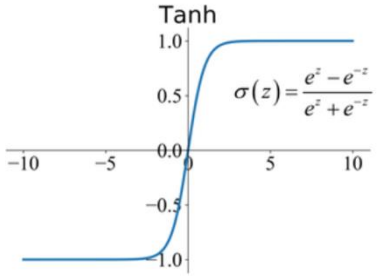
Full connection.

(actually just another special convolution)

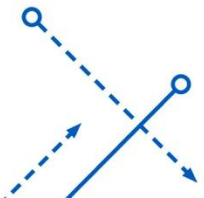
amplitude scale

$$f(a) = A \cdot \tanh(Sa)$$

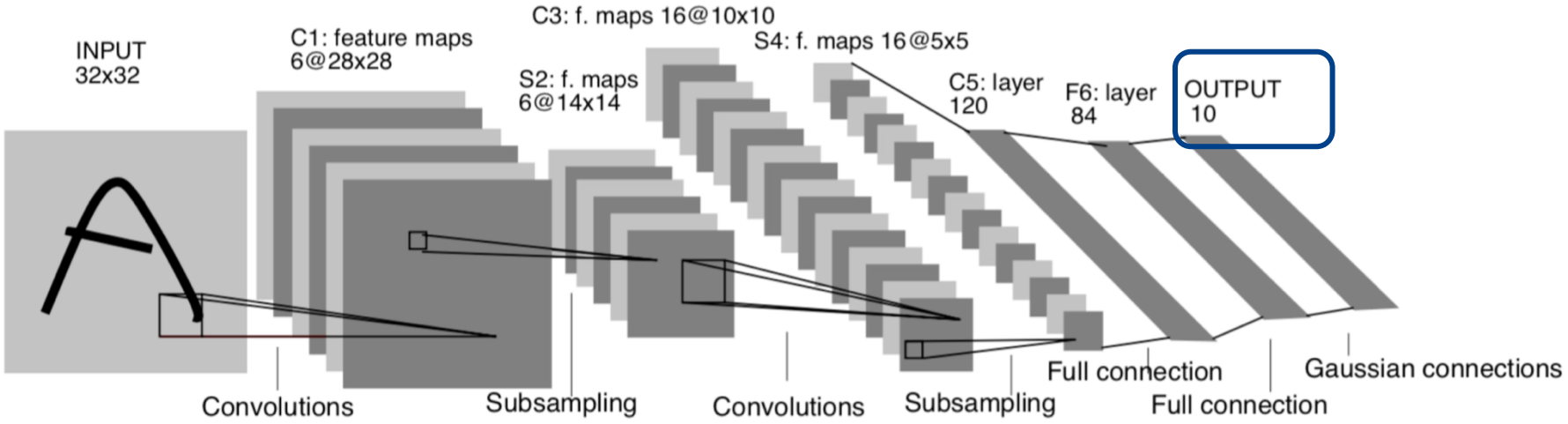
weighted sum



How many perceptrons to generate F6?



Output layer



$$y_i = \sum_{j}^{84} (x_j - w_{ij})^2$$

single class output

FC6

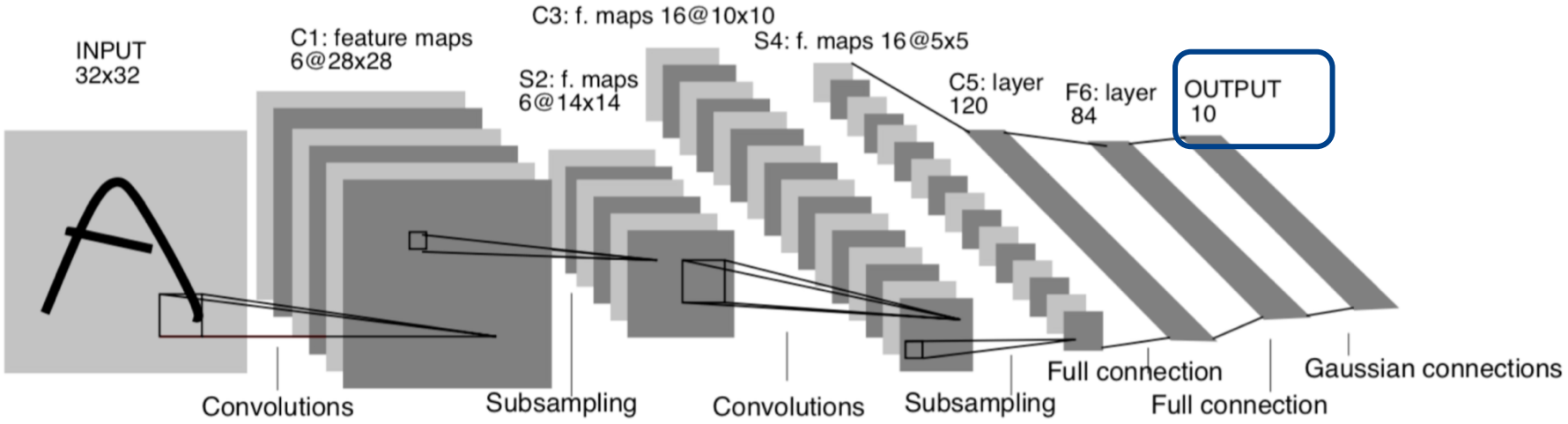
parameter set manually (-1, +1)

What is the range of i?

What happens when x matches w?

How do you set w?

Output layer



$$y_i = \sum_{j}^{84} (x_j - w_{ij})^2$$

single class output

FC6

w is set manually to match ASCII character

$$w =$$



White: -1
Black: +1

LeNet 5 is trained to draw characters!

More frequently used loss function

- Cross-entropy loss

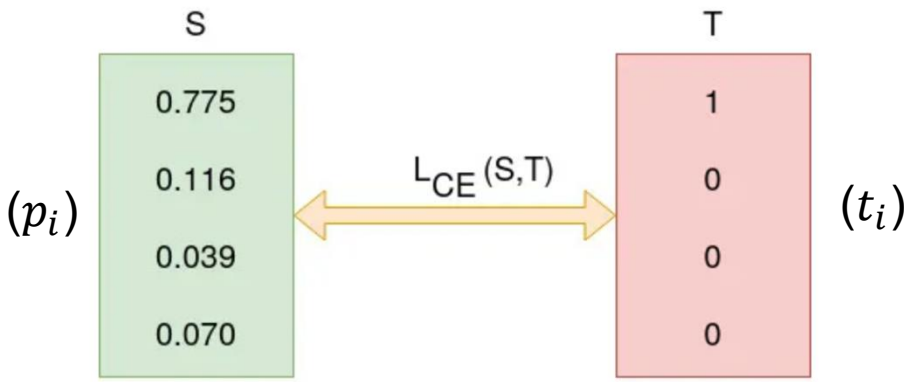
Softmax function

$$p\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

$$L_{\mathrm{CE}} = -\sum_{i=1}^{n} t_i \log(p_i), \quad \text{for n classes,}$$

where t_i is the truth label and p_i is the Softmax probability for the ith class.

- Target as one-hot vector



Logits(S) and one-hot encoded truth label(T) with Categorical Cross-Entropy loss function used to measure the 'distance' between the predicted probabilities and the truth labels. (Source: Author)

Why do we call it cross-entropy?

- We need to define Information.
  - High Information (surprising): low probability
  - Low Information (unsurprising): high probability
- We need map probability $(0, 1)$ to information $(\infty, 0)$
- So, we define information as

$$I(x) = -\log p(x)$$

- Recall the expectation of an event x.

$$\int xp(x)\,dx \quad \text{or} \quad \sum x_i p(x_i)$$

- The **entropy** (expected information) is

$$-\int p(x)\log p(x)\,dx \quad \text{or} \quad -\sum_i p(x_i)\log p(x_i)$$

Cross-entropy

- Cross-entropy: change $p(x_i)$ or p_i in **entropy** to distribution t_i:

$$L_{\mathrm{CE}} = -\sum_{i=1}^{n} t_i \log(p_i), \quad \text{for n classes,}$$

where t_i is the truth label and p_i is the Softmax probability for the ith class.

- Expected information of p_i with respect to distribution t_i.

- A measure of the **difference between two probability distributions**, typically between a predicted probability distribution and a true probability distribution.

Why is Cross-entropy often used?

- Easy to compute:
  - For binary classification:

$$L = -\left(y \log(p) + (1-y) \log(1-p)\right)$$

  - Gradient (sigmoid activation function):

$$\frac{\partial L}{\partial p} = p - y$$

  - Multi-class
    - p_c is the multiclass probability
    - y_c is one-hot vector

$$\frac{\partial L}{\partial p_c} = p_c - y_c$$

- Useful Gradient information:
  - If predictions are correct → $p_c - 1$, pushing p_c to 1.
  - If predictions are wrong → p_c , pushing p_c to 0.

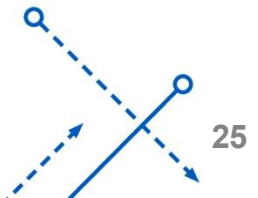
Why we prefer Cross-entropy loss?

- Fast Convergence:
  - Strong, informative feedback (larger gradients) when the model's predictions are confidently wrong
  - Enabling faster initial learning compared to other loss functions like Mean Squared Error (MSE).
- Avoids Vanishing Gradients:
  - The direct, linear nature of the (p-y) gradient prevents issues with vanishing gradients that can occur when using MSE with sigmoid activation, particularly when predictions are close to 0 or 1.
- Intuitive Update
  - The model is encouraged to adjust its parameters to make p closer to y.



Important Concepts

- Small filters capture common patterns
- Small filters can be reused and are efficient
- Multi-channel 2D Convolution
- Average pooling
- Convolution+Pooling+Non-linearity Module



AlexNet (2012)

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

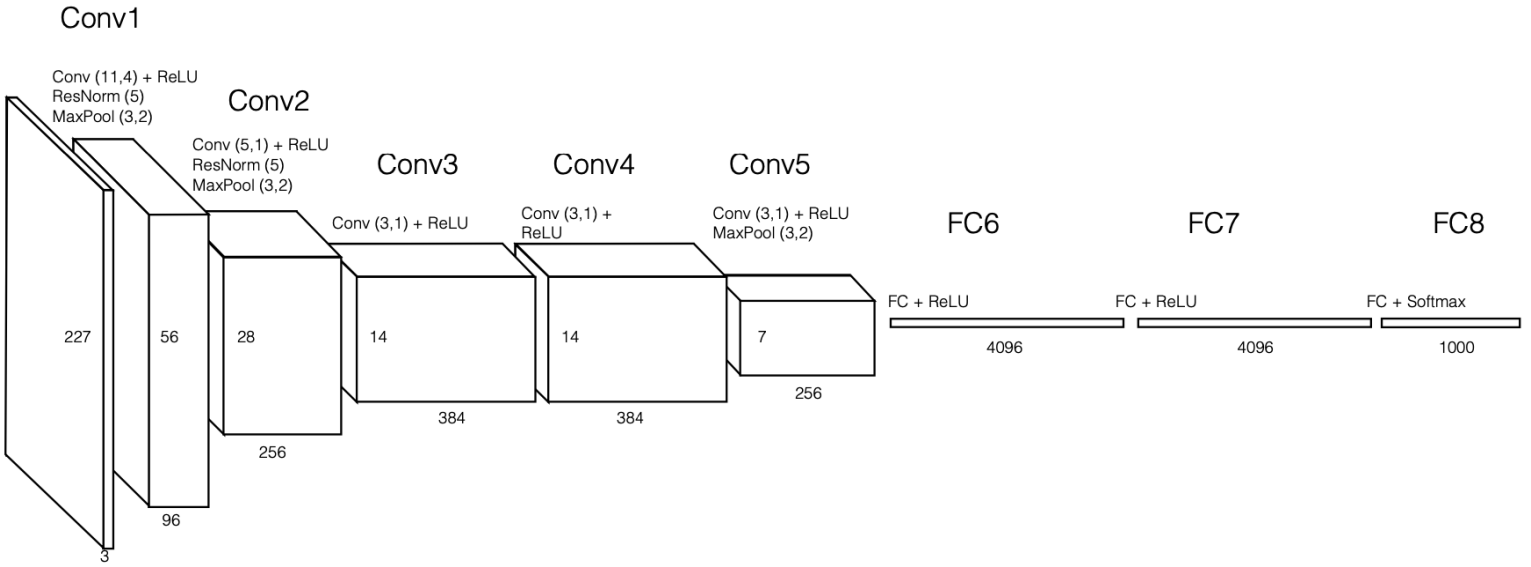
Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called "dropout" that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

Model	Top-1	Top-5
Sparse coding [2]	*47.1%*	*28.2%*
SIFT + FVs [24]	*45.7%*	*25.7%*
CNN	**37.5%**	**17.0%**

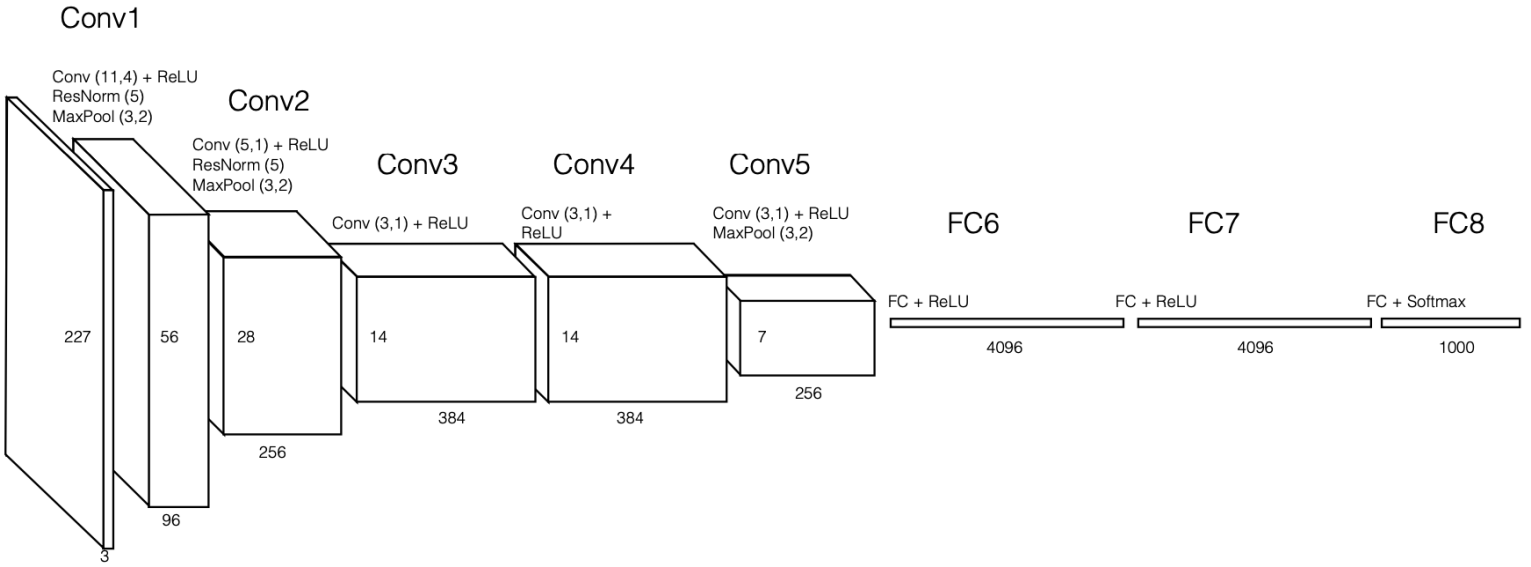
Table 1: Comparison of results on ILSVRC-2010 test set. In *italics* are best results achieved by others.

AlexNet structure



- Conv1 (a **convolution**, ReLU, ResNorm, max pooling)
  - Convolution with size? 3 x 11 x 11
  - How many filters in Conv1? 96
  - What is the size of the response map after Conv1? 56
  - What is the stride of the convolutions? 4
- Shorthand notation: Conv(size:11, stride:4)

AlexNet structure



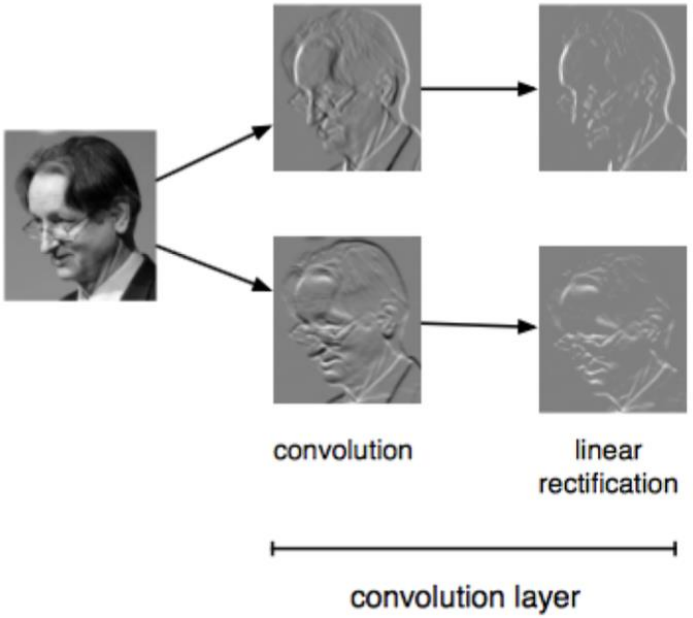
- Conv1 (a convolution, **ReLU**, ResNorm, max pooling)
- Rectified Linear Unit (ReLU)

$$f(x) = \max(0, x)$$ Pixel-wise operator

- Keeps all of the positive activations and throws away all negative!
- Why would we want a non-linear activation function like this?

Why use ReLU

- Two linear functions (convolutions) is the same as one linear function (convolution).
  - ReLU adds a non-linearity and is more powerful when stacking convolutions.
- Makes gradients sparse (1): Helps avoid vanishing gradients (recall the chain rule).
- Really speeds up convergence of gradient descent



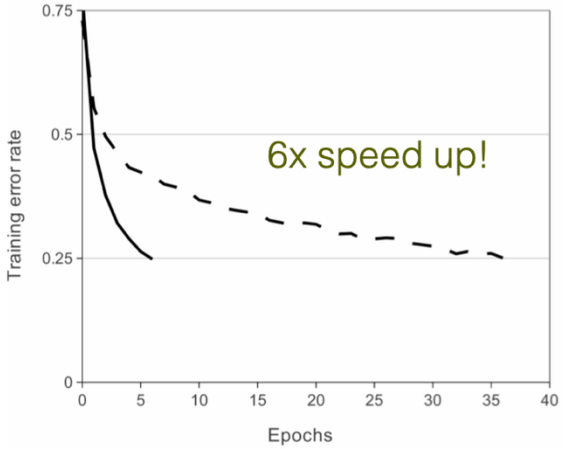
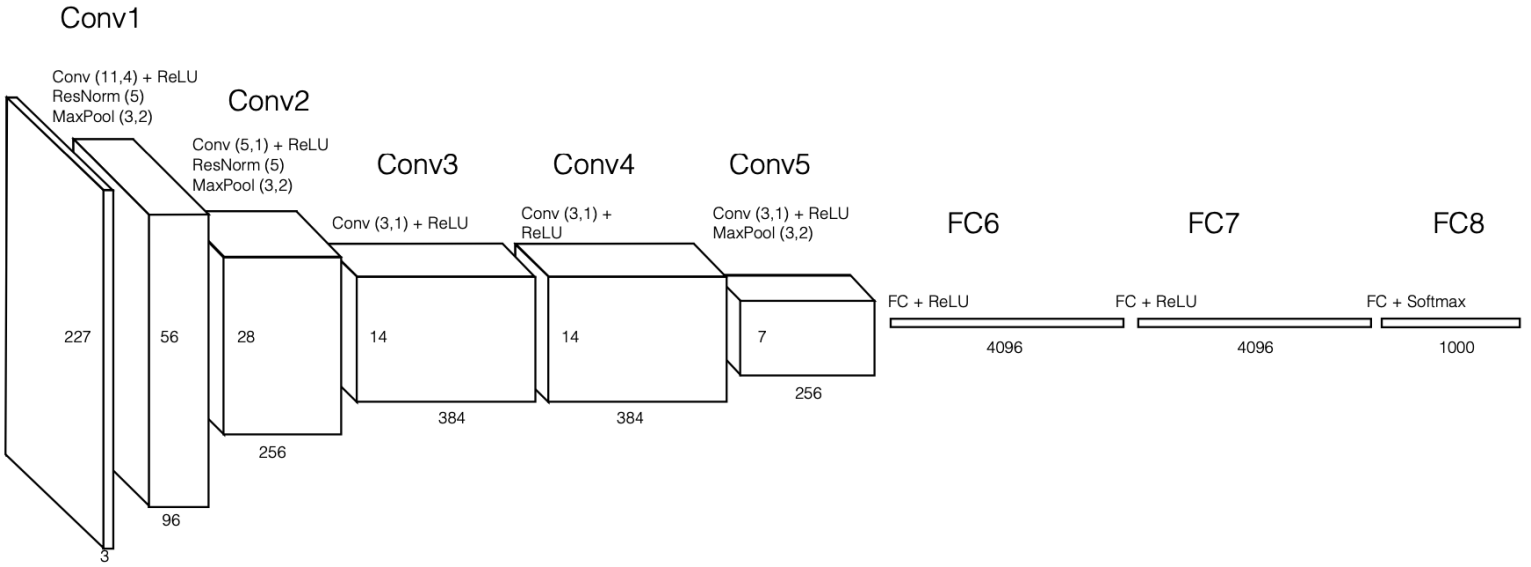


Figure 1: A four-layer convolutional neural network with ReLUs **(solid line)** reaches a 25% training error rate on CIFAR-10 six times faster than an equivalent network with tanh neurons **(dashed line)**. The learning rates for each net-

AlexNet structure



- Conv1 (a convolution, ReLU, **ResNorm**, max pooling)

Local response normalization

$$b^i_{x,y} = \frac{a^i_{x,y}}{\left(k + \alpha \sum_{j=-2}^{2} (a^j_{x,y})^2\right)^{\beta}}$$

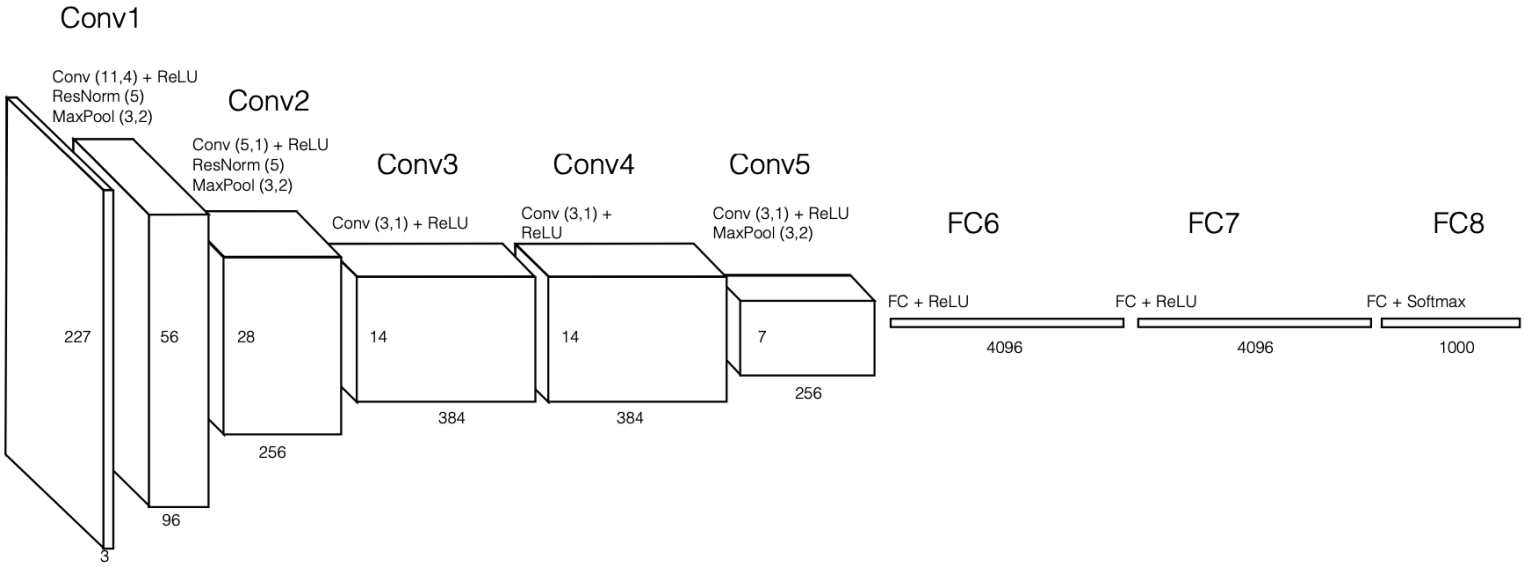
channel

Activation location

offset (2) Scale (10^{-4}) 5 channel window Non-linear scaling (0.75)

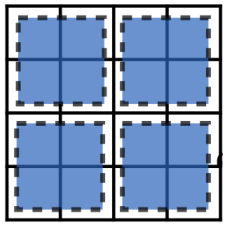
1.2~1.4% reduction in error rate

AlexNet structure



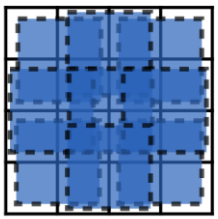
- Conv1 (a convolution, ReLU, normalization, **max pooling**)
- Overlap maximum pooling.

Traditional Pooling



Size: 2 x 2
Stride: 2
'MaxPool (2,2)'

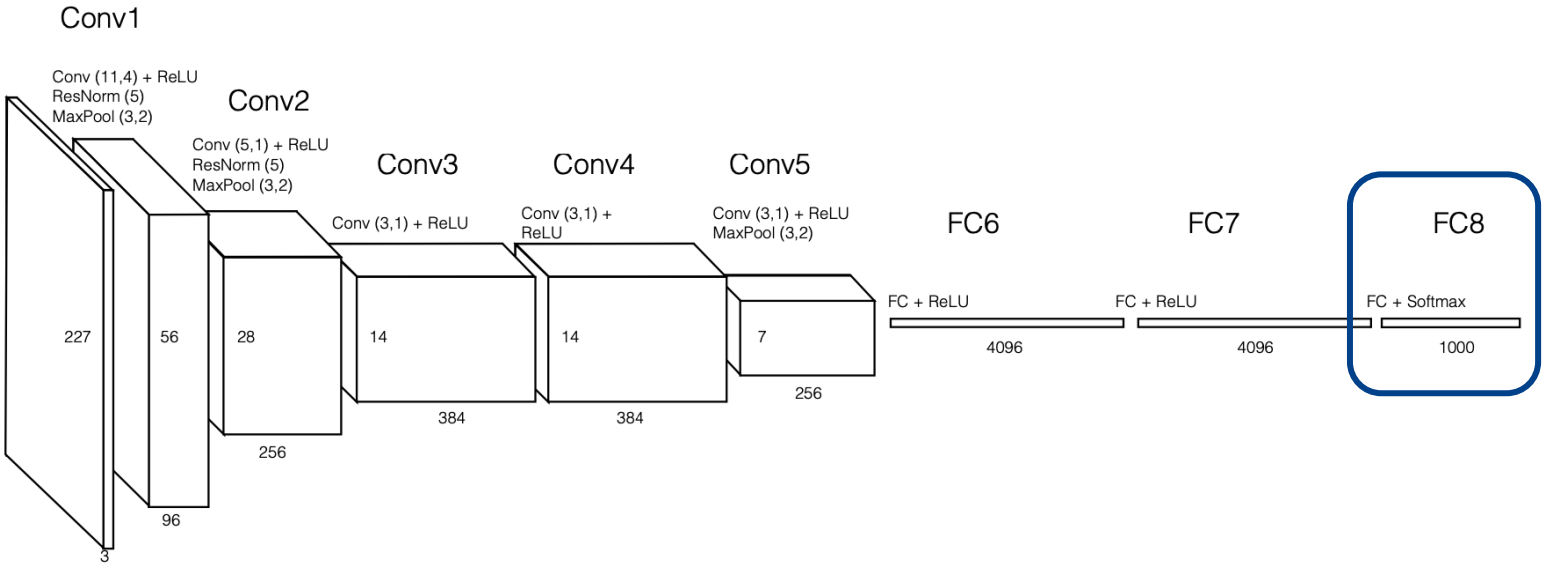
Overlapped Pooling



Size: 2 x 2
Stride: 1
'MaxPool (2,1)'

0.3~0.5% reduction in error rate

AlexNet structure



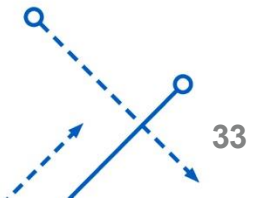
- Softmax

$$y_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

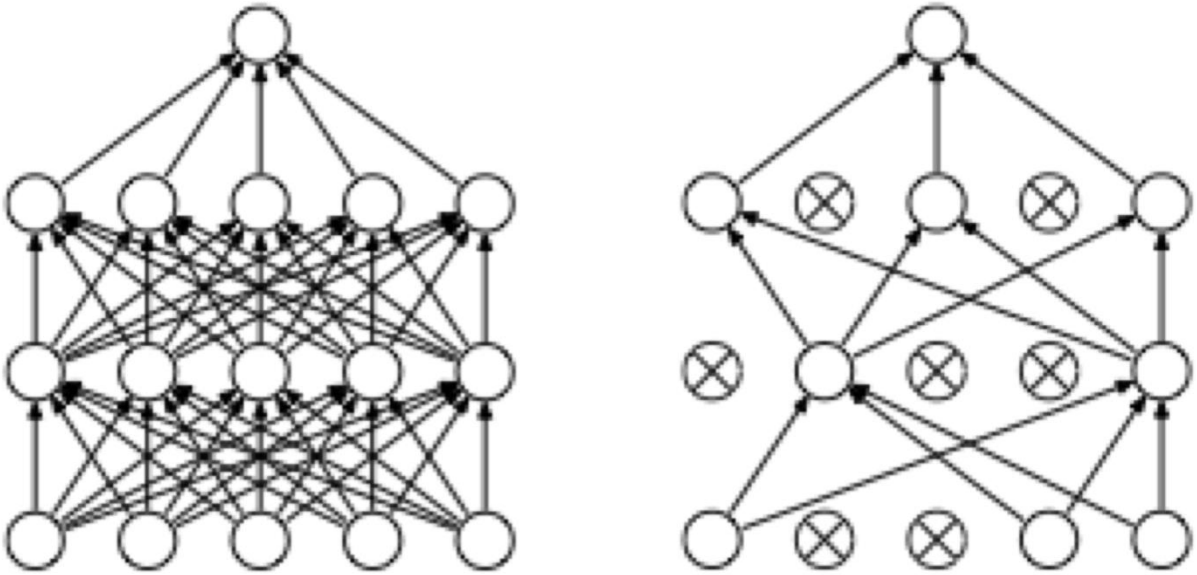
- Final softmax converts FC output values to probability

Tricks for Training AlexNet

- Dropout
- Data augmentation
- SGD with momentum



Dropout

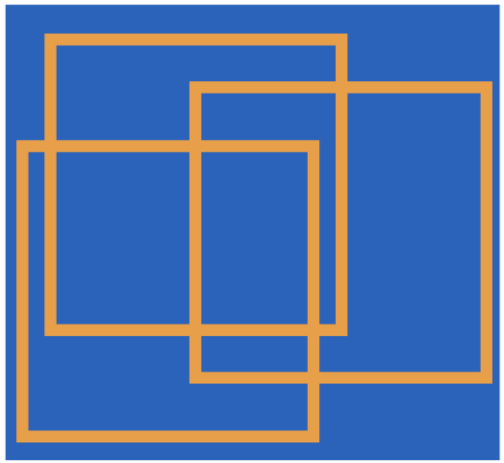


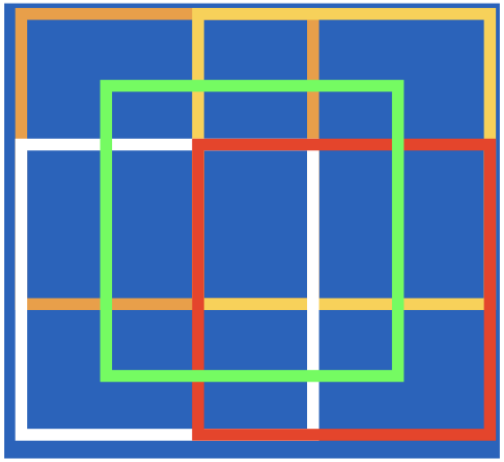
(a) Standard Neural Net (b) After applying dropout.

- During training certain nodes are randomly dropped out in FC.
- Forces the network to learn with less parameters.
  - Reduces overfitting (doubles training time).
- Has the effect of model averaging.
  - Use all nodes at test time .

Data Augmentation

- Random translation and horizontal flipping.
- A test time, prediction is averaged over 10 patches.
  - corners, centers, and flips.





- Color augmentation
  - Changes the color and intensity of the image
    - Reduces top-1 error rate by 1%
- Prevents overfitting

SGD with momentum

- Gradient Update

update equation

$$w_{i+1} := w_i + v_{i+1}$$

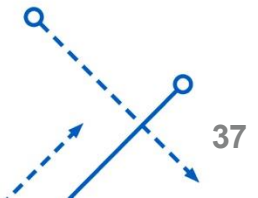
momentum: 'keep going'; weight decay; learning rate; expected gradient of the loss; mini-batch

$$v_{i+1} := 0.9 \cdot v_i - 0.0005 \cdot \epsilon \cdot w_i - \epsilon \cdot \left\langle \frac{\partial L}{\partial w}\Big|_{w_i} \right\rangle_{D_i}$$

- Momentum helps push gradients vectors to the right directions, thus leading to **faster converging**
- Weight decay improves the convergence by **encouraging smaller weights (like gradients of regularization)**.

Important Concepts

- Overlapped Pooling
- Response Normalization
- ReLU
- Softmax
- Training trick: Dropout
- Training trick: Data Augmentation
- Training trick: momentum and decay
- 2 GPUs, 6 days to train



VGG Net (2015)

Published as a conference paper at ICLR 2015

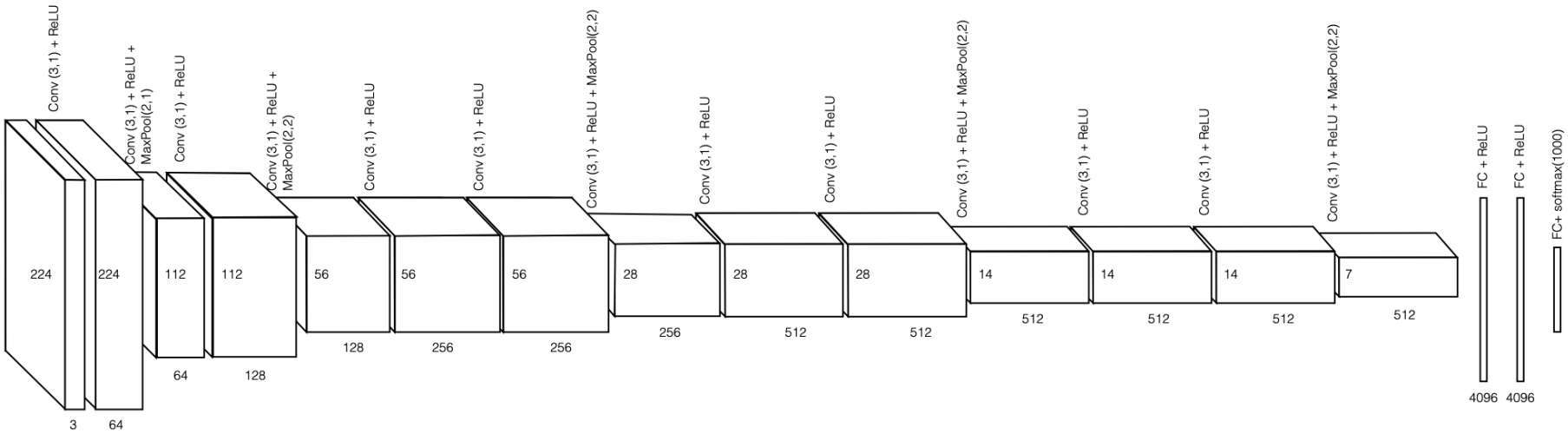
VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* **& Andrew Zisserman+**
Visual Geometry Group, Department of Engineering Science, University of Oxford
{karen,az}@robots.ox.ac.uk

ABSTRACT

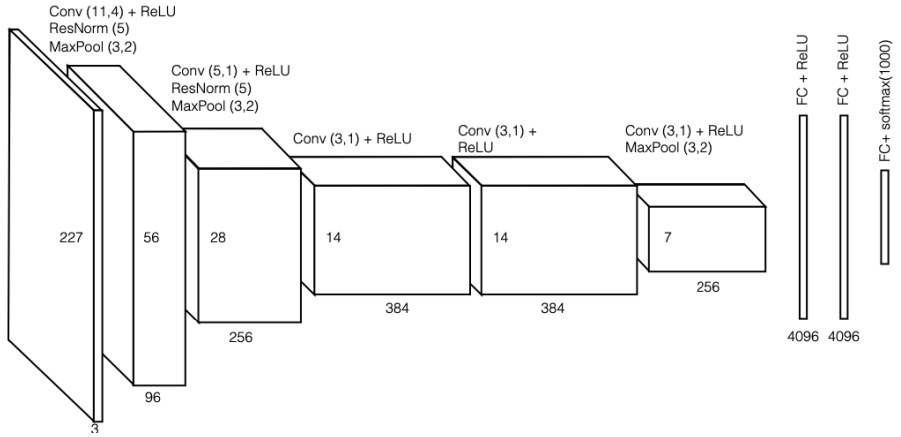
In this work we investigate the effect of the convolutional network depth on its accuracy in the large-scale image recognition setting. Our main contribution is a thorough evaluation of networks of increasing depth using an architecture with very small (3×3) convolution filters, which shows that a significant improvement on the prior-art configurations can be achieved by pushing the depth to 16–19 weight layers. These findings were the basis of our ImageNet Challenge 2014 submission, where our team secured the first and the second places in the localisation and classification tracks respectively. We also show that our representations generalise well to other datasets, where they achieve state-of-the-art results. We have made our two best-performing ConvNet models publicly available to facilitate further research on the use of deep visual representations in computer vision.

VGG-16


Conv (3,1) + ReLU
Conv (3,1) + ReLU + MaxPool(2,1)
Conv (3,1) + ReLU
Conv (3,1) + ReLU + MaxPool(2,2)
Conv (3,1) + ReLU
Conv (3,1) + ReLU
Conv (3,1) + ReLU + MaxPool(2,2)
Conv (3,1) + ReLU
Conv (3,1) + ReLU
Conv (3,1) + ReLU + MaxPool(2,2)
Conv (3,1) + ReLU
Conv (3,1) + ReLU
Conv (3,1) + ReLU + MaxPool(2,2)
FC + ReLU
FC + ReLU
FC+ softmax(1000)
224
224
112
112
56
56
56
28
28
28
14
14
14
7
3
64
64
128
128
256
256
256
512
512
512
512
512
4096
4096

Improvements?

AlexNet

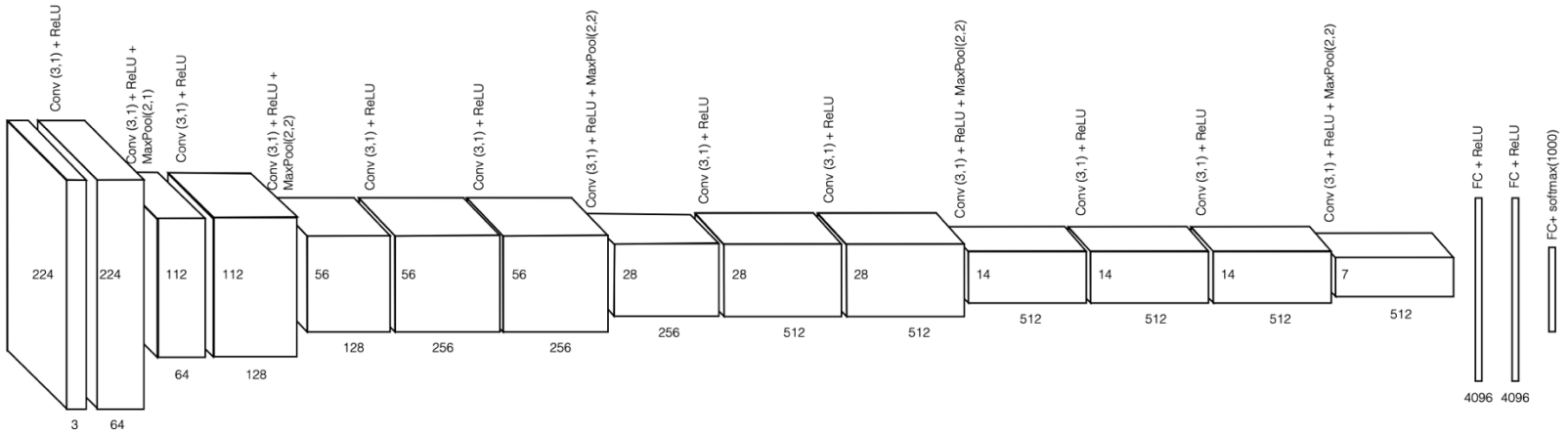


8 layers

deeper!

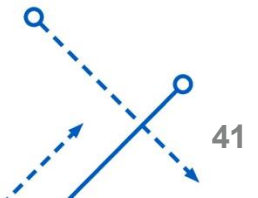
VGG-16

16 layers



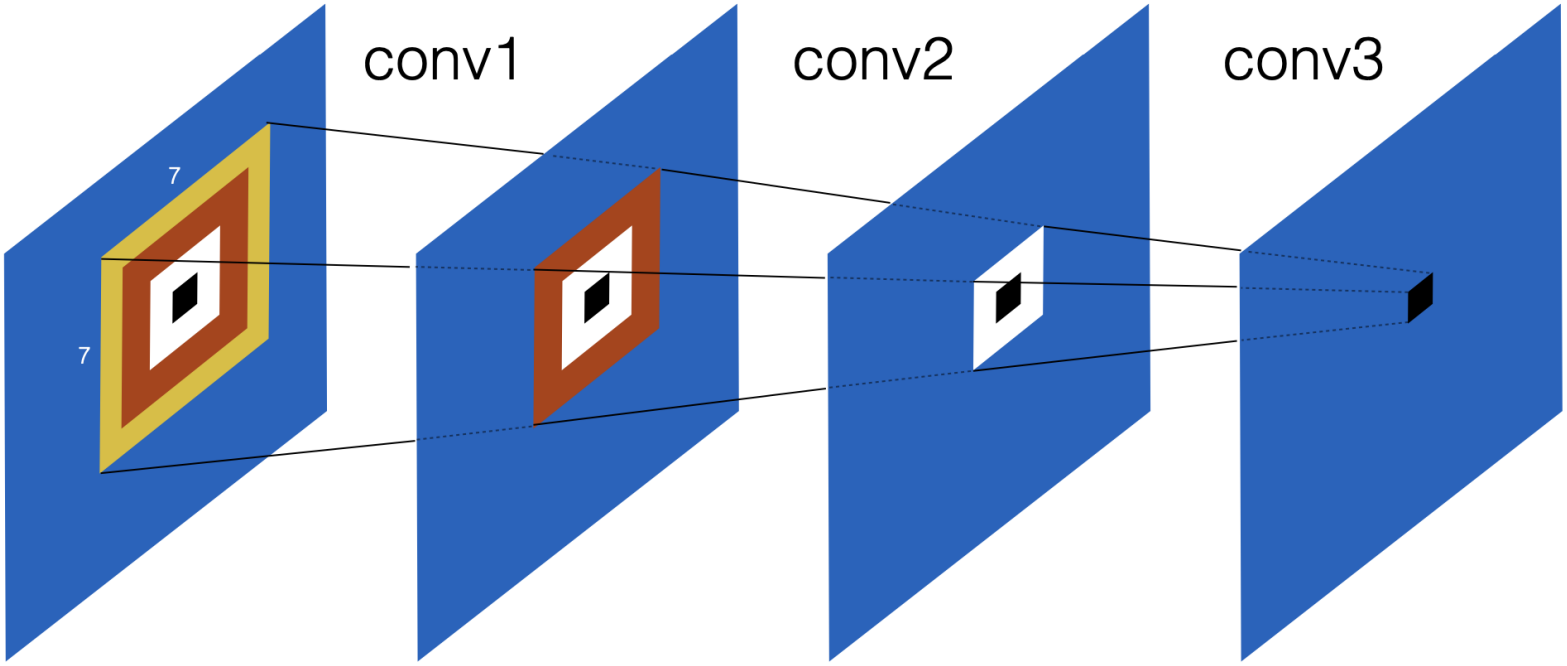
VGG-16

- Convolutions all standardized to 3×3
- Convolutions always followed by ReLU
- Stack several convolutions at the same resolution
- Downsample by 2, increase channels by 2
- No local response normalization



Stacking convolutions

- What is the receptive field of 3 stacks of 3x3 convolutions?



Stacking convolutions

- What do you gain by stacking convolutions?
  - Decision function is more discriminative.

Two linear functions

$$b \cdot (a \cdot x)$$

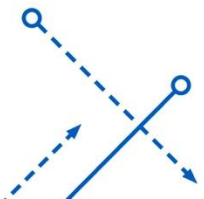
Same as just one linear function

Two non-linear functions

$$F(b \cdot F(a \cdot x))$$

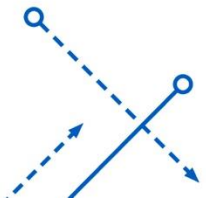
Can capture more complex decision boundary

- Decrease # parameters for same receptive field.
  - Three $3 \times 3 \times C$ convolution layers: $27C$ parameters.
  - One $7 \times 7 \times C$ convolution layer: $49C$ parameters.



Tricks for Training VGG

- Pre-training
- Data augmentation (crops, flips, color, scale)
- Dropout
- SGD with momentum
- 4 GPUs, 21 days, to train



ResNet (2016)

This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the version available on IEEE Xplore.

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

Abstract

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers—8× deeper than VGG nets [40] but still having lower complexity. An ensemble of these residual nets achieves 3.57% error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers.

The depth of representations is of central importance for many visual recognition tasks. Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset. Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions[1], where we also won the 1st places on the tasks of ImageNet detection, ImageNet local-

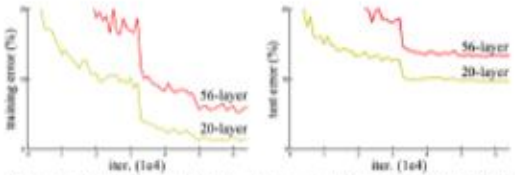


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

greatly benefited from very deep models.

Driven by the significance of depth, a question arises: *Is learning better networks as easy as stacking more layers?* An obstacle to answering this question was the notorious problem of vanishing/exploding gradients [14, 1, 8], which hamper convergence from the beginning. This problem, however, has been largely addressed by normalized initialization [23, 8, 36, 12] and intermediate normalization layers [16], which enable networks with tens of layers to start converging for stochastic gradient descent (SGD) with back-propagation [22].

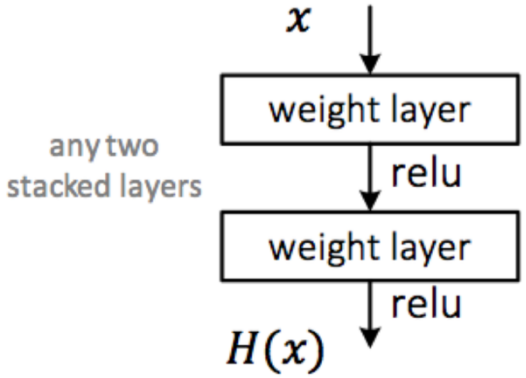
When deeper networks are able to start converging, a

Deep Residual Learning

$H(x)$ is any desired mapping,

hope the 2 weight layers fit $H(x)$

- Plaint net



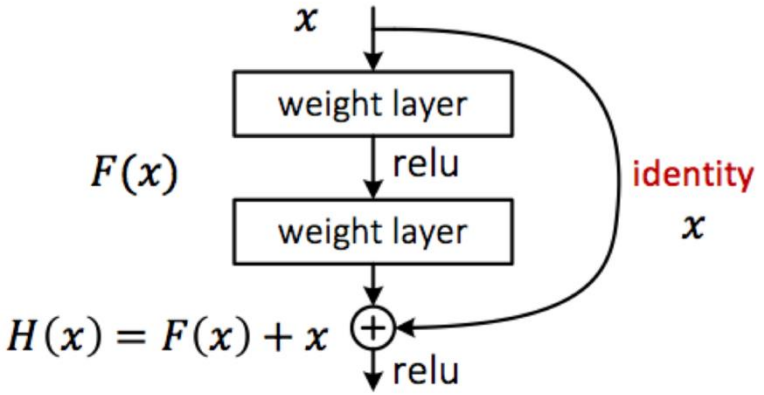
$H(x)$ is any desired mapping,

~~hope the 2 weight layers fit $H(x)$~~

hope the 2 weight layers fit $F(x)$

let $H(x) = F(x) + x$

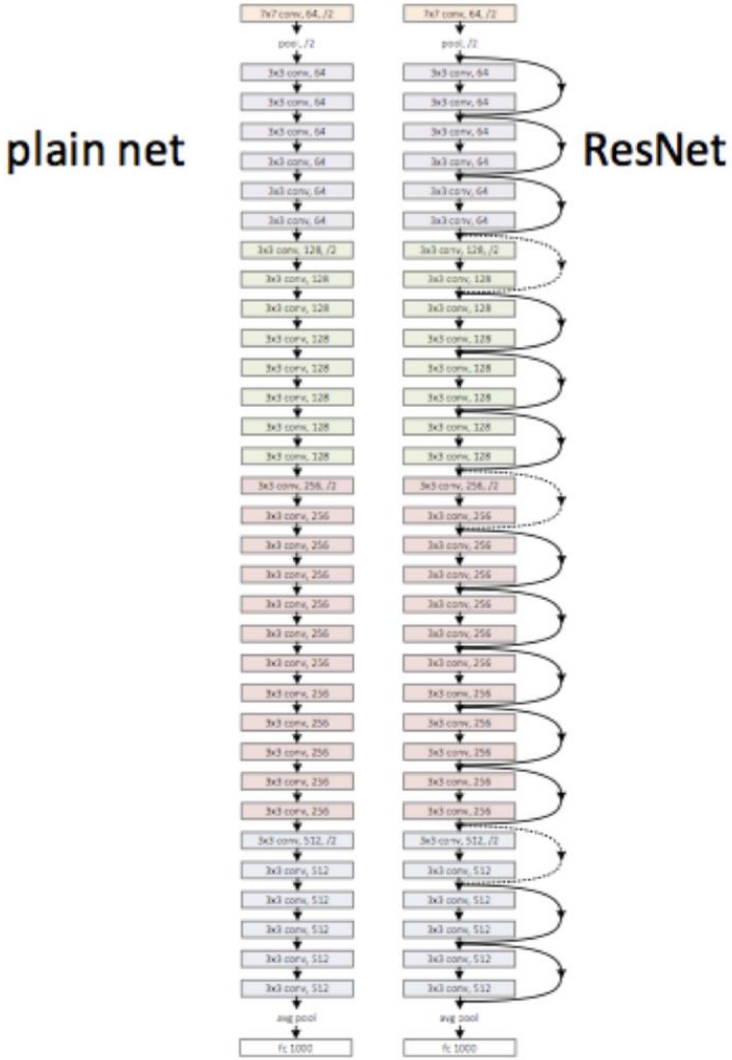
- Residual net



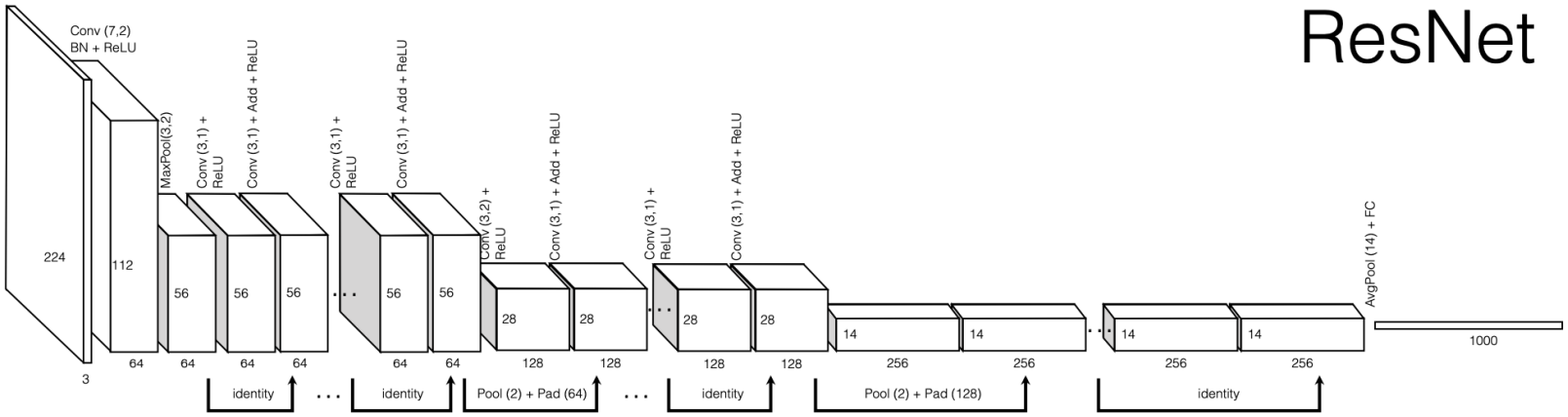
- If identity were optimal, easy to set weights as 0
- If optimal mapping is closer to identity, easier to find small fluctuations

Network design

- Simple design; just deep!



ResNet

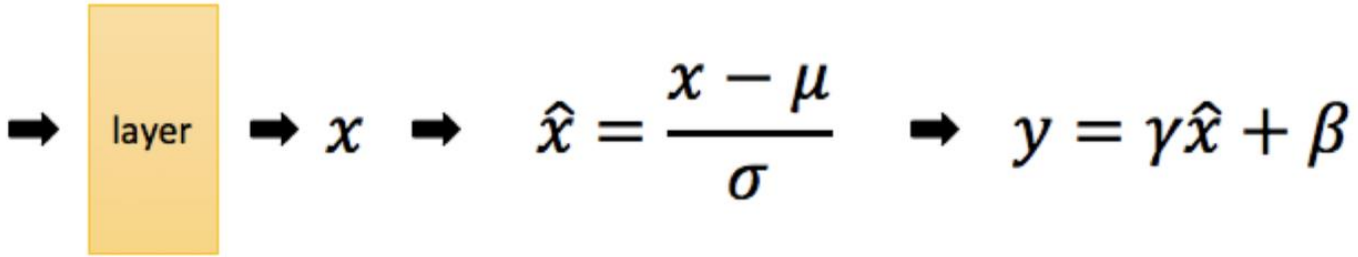


- Convolution + Batch Normalization + ReLU
- 7 x 7 convolution just for the first layer
- Stride of 2 reduces the resolution in half
- Batch Normalization to scale the responses
  - ReLU non-linearity

Batch Normalization

- A really small step makes the training time really long
- Instead, for each training mini-batch...

Batch Normalization (BN)

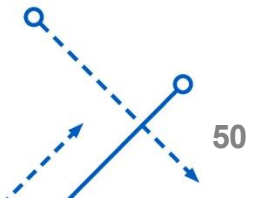


- μ: mean of x in mini-batch
- σ: std of x in mini-batch
- γ: scale
- β: shift

- μ, σ: functions of x, analogous to responses
- γ, β: parameters to be learned, analogous to weights

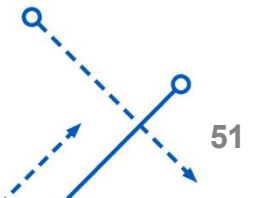
Batch Normalization

- Normalizing input
  - LeCun et al 1998 "Efficient Backprop"
- BN: normalizing each layer, for each mini-batch
  - Greatly accelerate training
  - Less sensitive to initialization
  - Improve regularization



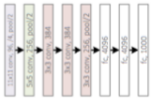
ResNet

- 1st places in all five main tracks
- ImageNet Classification: "Ultra-deep" 152-laver nets
- ImageNet Detection: 16% better than 2nd
- ImageNet Localization: 27% better than 2nd
- COCO Detection: 11% better than 2nd
- COCO Segmentation: 12% better than 2nd

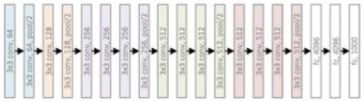


Going deeper and deeper

AlexNet



VGG-19



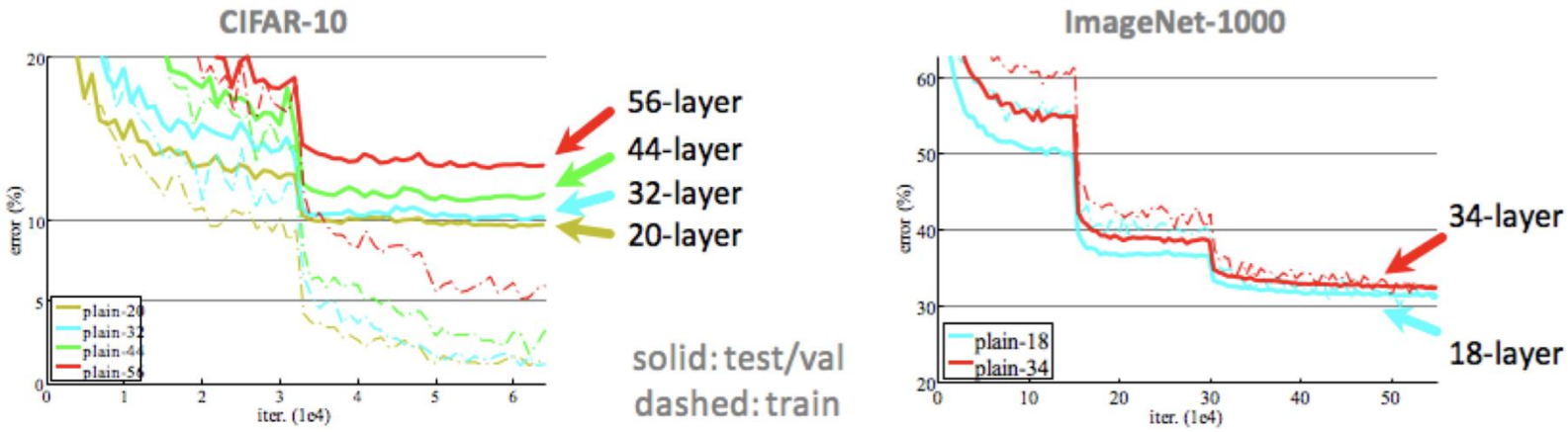
ResNet-152



Going deeper and deeper

- Deeper is better so just stack more convolutions.
- Right?

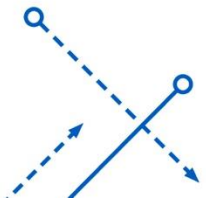
NO!



- Deeper networks have higher training error!

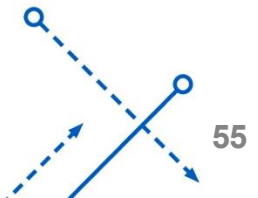
Going deeper and deeper

- Expectation
  - If adding extra identity layers on top of shallow one.
  - Training error should be the same not worse.
  - Deeper models shouldn’t have higher training error.
- Reality
  - Solvers (gradient descent) cannot find the solution.
- Solution
  - Start with the identity solution and then tune parameters.
  - This is residual learning!



Important Concepts

- Residual Learning
- Skip connections



Vision Transformer (ViT)

Published as a conference paper at ICLR 2021

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy[*,†], **Lucas Beyer**[*], **Alexander Kolesnikov**[*], **Dirk Weissenborn**[*], **Xiaohua Zhai**[*], **Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby**[*,†]

[*]equal technical contribution, [†]equal advising

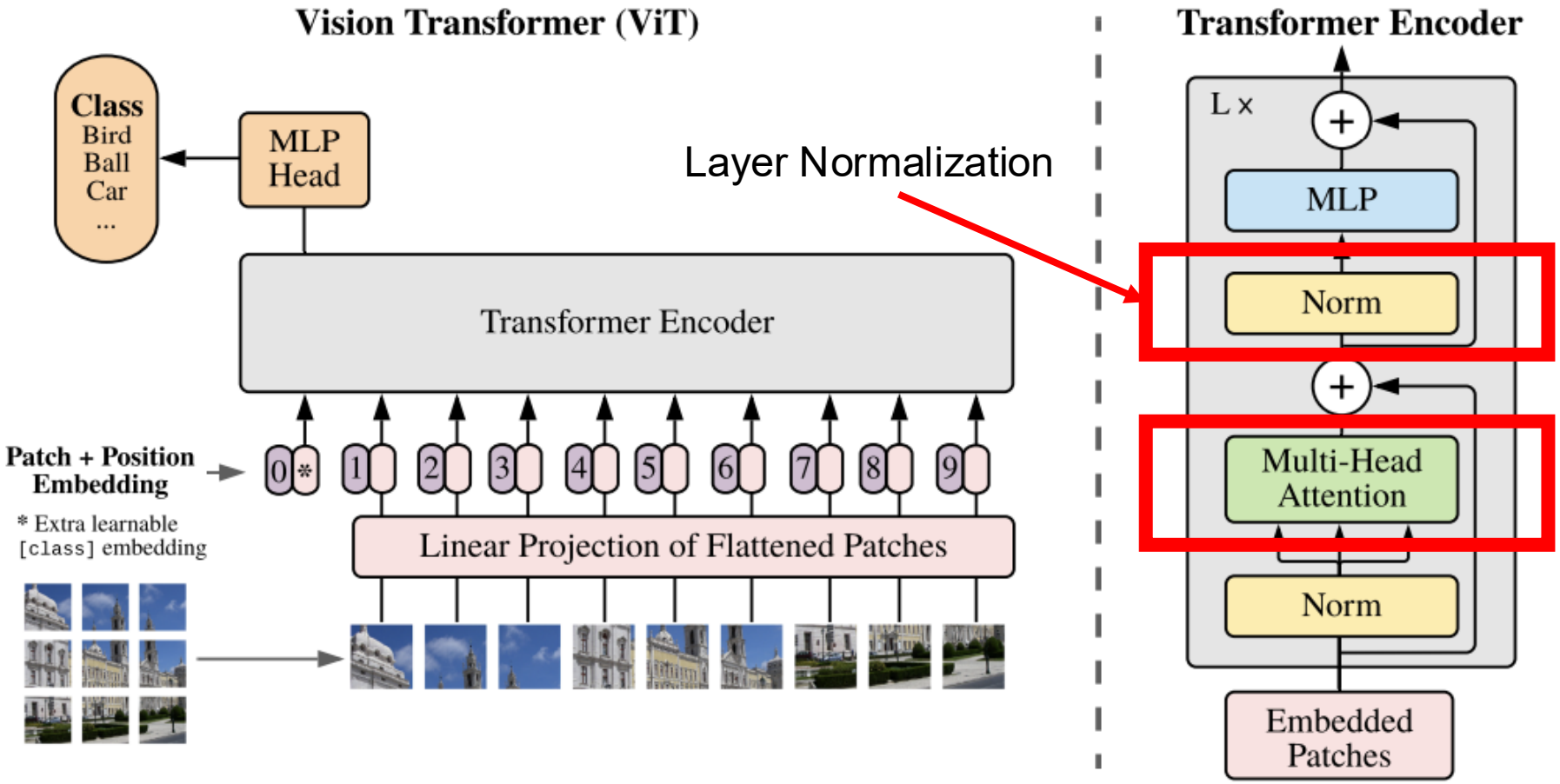
Google Research, Brain Team

{adosovitskiy, neilhoulsby}@google.com

ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.[1]

Vision Transformer (ViT)



Attention

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

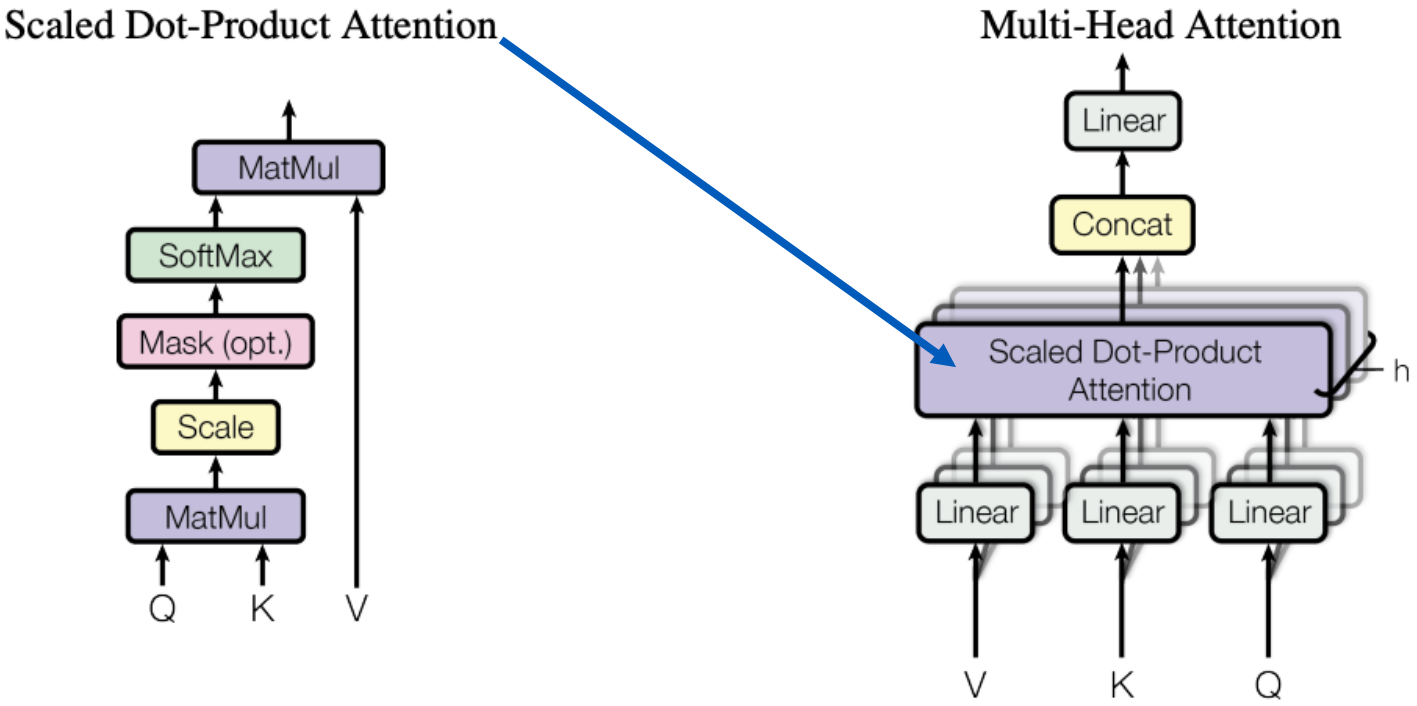
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

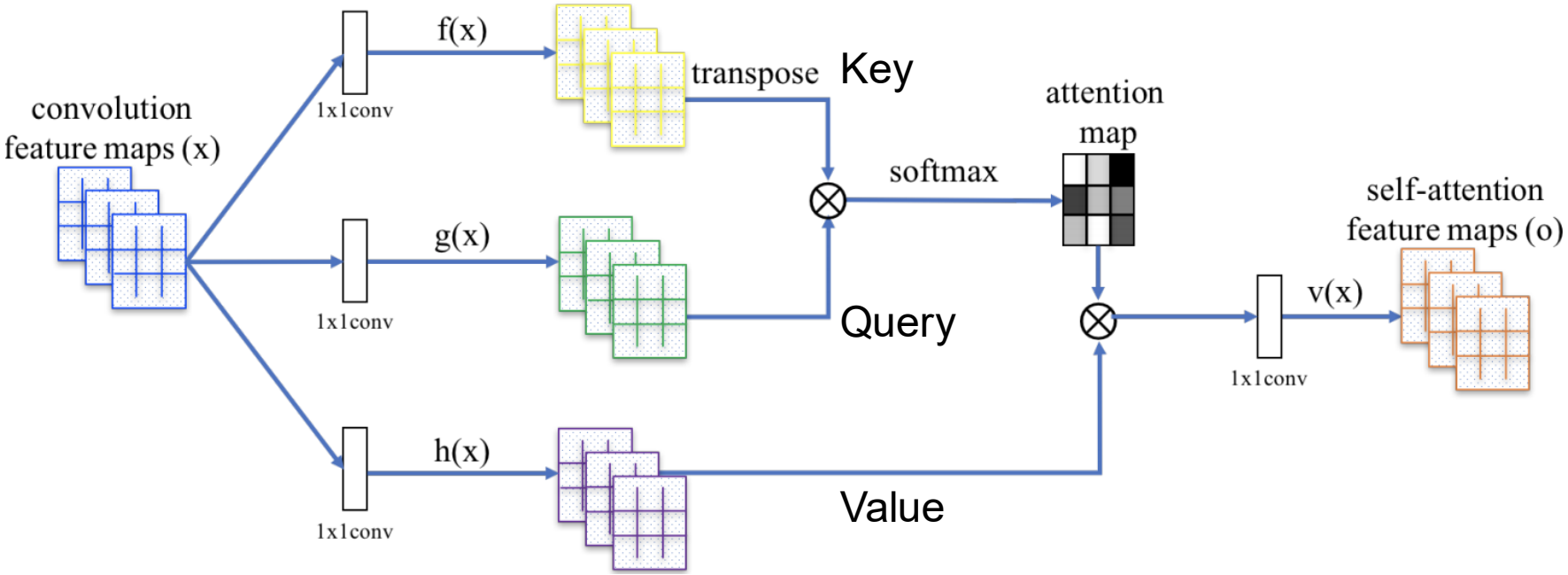
Attention

- Query, Key, Value

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



Self-attention in ViT



Layer vs Batch Normalization

- Layer Normalization

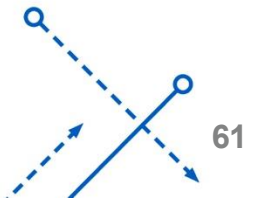
$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

Normalizes the inputs across all features for **individual training examples**.

- Batch Normalization

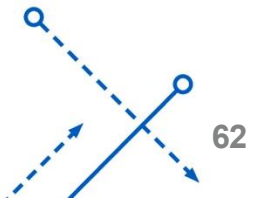
$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}} * \gamma + \beta$$

Normalizes the inputs of a layer for **each mini-batch**.



Important Concepts

- Layer Normalization
- Self-attention



Conclusion

- Network Structures
  - LetNet-5
  - ResNet
  - Vision Transformer
- Training Techniques
  - SGD, why can it converge intuitively?
  - Cross-Entropy Loss
  - Data Augmentation
  - Drop Out
  - Normalization

